Reviewer Pack — SOS Moment Matrix Rank and Max-CUT Integrality Gap

Entry 8d60c087478f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 20:31:12 UTC

SOS Moment Matrix Rank and Max-CUT Integrality Gap

Entry ID: `8d60c087478f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-10 20:31:12 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): SOS Degree for Max-CUT

Statement:

> For a random Max-CUT instance on n variables, the rank of the moment matrix in the d-th level of the SOS hierarchy is at least $\Omega(n/d^2)$, and this bound is tight for instances with high expansion properties.

Rationale (proposer's reasoning):

> The rank of the moment matrix captures the complexity of the polynomial constraints. High expansion in Max-CUT instances may require higher rank to capture the necessary inequalities, linking algebraic structure to SOS degree.

Taxonomy category: `SOS_DEGREE` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `210c1d80126c8054`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from itertools import combinations

def generate_max_cut_instance(n):
    G = {i: set() for i in range(n)}
    edges = list(combinations(range(n), 2))
    for u, v in edges:
        if random.random() < 0.5:
            G[u].add(v)
            G[v].add(u)
    return G

def moment_matrix(G, d):
    n = len(G)
    M = [[0] * (n + d) for _ in range(n + d)]
    for i in range(n):
        M[i][i] = 1
        for j in range(i + 1, n):
            if j in G[i]:
                M[i][j] = -1
                M[j][i] = -1
    for k in range(2, d + 1):
        for i in range(n):
            for j in range(i + 1, n):
                if j in G[i]:
                    M[n+k-1][n+j] = -1
    return M

def rank(matrix):
    m, n = len(matrix), len(matrix[0])
    matrix = [row[:] for row in matrix]
    pivot_row = 0
    for i in range(n):
        if pivot_row >= m:
            break
        max_abs = abs(matrix[pivot_row][i])
        max_row = pivot_row
        for j in range(pivot_row + 1, m):
            if abs(matrix[j][i]) > max_abs:
                max_abs = abs(matrix[j][i])
                max_row = j
        if max_abs == 0:
            continue
        matrix[pivot_row], matrix[max_row] = matrix[max_row], matrix[pivot_row]
```

```
        for j in range(n):
            matrix[pivot_row][j] /= matrix[pivot_row][i]
        for j in range(m):
            if j != pivot_row and abs(matrix[j][i]) > 1e-9:
                factor = matrix[j][i]
                for k in range(n):
                    matrix[j][k] -= factor * matrix[pivot_row][k]
        pivot_row += 1
    return sum(1 for row in matrix if any(abs(x) > 1e-9 for x in row))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    G = generate_max_cut_instance(n)
    d = min(10, n - 2)
    M = moment_matrix(G, d)
    rank_M = rank(M)
    metric_value = (rank_M * d**2) / n
    conjecture_holds = metric_value >= 1.0
    counterexample = "" if conjecture_holds else "mapping_undefined"
    return {
        "metric_name": "Rank of Moment Matrix",
        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2**i - 1 for i in range(5, 3
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] and r["counterexample"] == "mapping_undefined" for r in res
        print("RESULT: INCONCLUSIVE mapping_undefined")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
```

```
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f687d6d.py", line 90, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f687d6d.py", line 73, in run_trial
    M = moment_matrix(G, d)
        ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f687d6d.py", line 39, in moment_matrix
    M[n+k-1][n+j] = -1
    ~~~~~~~~~^^^^^
IndexError: list assignment index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with IndexError, preventing data collection. The pre-registered support condition cannot be evaluated without valid results. | next: Debug the moment_matrix implementation to fix index out-of-range errors and re-run tests

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	50440
2	preregistration	ollama_remote	qwen3:8b	0	0	24062
3	novelty	ollama_remote	qwen3:8b	0	0	23723
4	novelty	ollama_remote	qwen3:8b	0	0	16652
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12594
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10788
7	judge	ollama_remote	qwen3:8b	0	0	16046

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 154305 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/8d60c087478f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8d60c087478f.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/8d60c087478f.tar.gz` (if generated)